

Assignment 9: Spatial Analysis in R

Maeve Gualtieri-Reed

OVERVIEW

This exercise accompanies the lessons in Environmental Data Analytics (ENV872L) on spatial analysis.

Directions

1. Rename this file `<FirstLast>_A09_SpatialAnalysis.Rmd` (replacing `<FirstLast>` with your first and last name).
2. Change “Student Name” on line 3 (above) with your name.
3. Use the lesson as a guide. It contains code that can be modified to complete the assignment.
4. Work through the steps, **creating code and output** that fulfill each instruction.
5. Be sure to **answer the questions** in this assignment document. Space for your answers is provided in this document and is indicated by the “>” character. If you need a second paragraph be sure to start the first line with “>”. You should notice that the answer is highlighted in green by RStudio.
6. When you have completed the assignment, **Knit** the text and code into a single **HTML** file.

DATA WRANGLING

Set up your session

1. Import libraries: tidyverse, sf, leaflet, here, and mapview
2. Execute the `here()` command to display the current project directory

#1.

```
library(tidyverse);library(here);
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.1      v tibble    3.2.1
## v lubridate  1.9.3      v tidyr     1.3.1
## v purrr      1.0.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
## here() starts at /home/guest/EDE_Fall2025
```

```
library(sf);library(mapview);library(leaflet);library(viridis)
```

```
## Linking to GEOS 3.10.2, GDAL 3.4.1, PROJ 8.2.1; sf_use_s2() is TRUE
## Loading required package: viridisLite
```

```
#2.  
here()
```

```
## [1] "/home/guest/EDE_Fall2025"
```

Read (and filter) county features into an sf dataframe and plot

In this exercise, we will be exploring stream gage height data in Nebraska corresponding to floods occurring there in 2019. First, we will import from the US Counties shapefile we've used in lab lessons, filtering it this time for just Nebraska counties. Nebraska's state FIPS code is 31 (as North Carolina's was 37).

3. Read the `cb_2018_us_county_20m.shp` shapefile into an sf dataframe, filtering records for Nebraska counties (State FIPS = 31)
4. Reveal the dataset's coordinate reference system
5. Plot the records as a map (using `mapview` or `ggplot`)

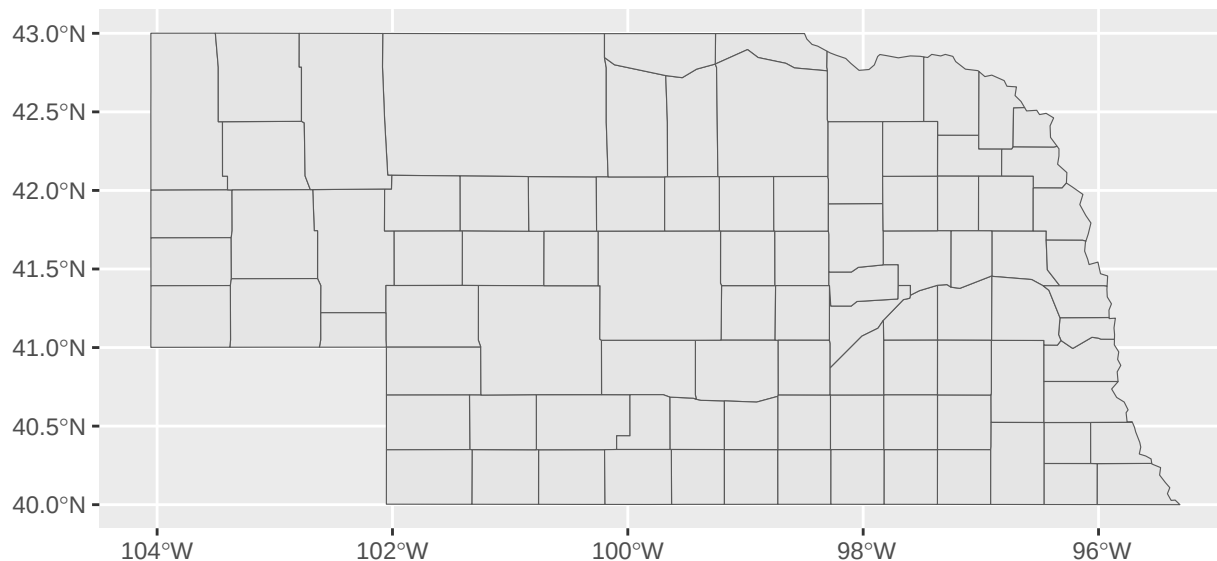
```
#3. Read in Counties shapefile into an sf dataframe, filtering for just NE counties  
Nebraska_county <- st_read(here('Data/Spatial/cb_2018_us_county_20m.shp')) %>%  
  filter(STATEFP == 31)
```

```
## Reading layer 'cb_2018_us_county_20m' from data source  
##   '/home/guest/EDE_Fall2025/Data/Spatial/cb_2018_us_county_20m.shp'  
##   using driver 'ESRI Shapefile'  
## Simple feature collection with 3220 features and 9 fields  
## Geometry type: MULTIPOLYGON  
## Dimension:      XY  
## Bounding box:   xmin: -179.1743 ymin: 17.91377 xmax: 179.7739 ymax: 71.35256  
## Geodetic CRS:   NAD83
```

```
#4. Reveal the CRS of the counties features  
st_crs(Nebraska_county)$epsg
```

```
## [1] 4269
```

```
#5. Plot the data  
ggplot() +  
  geom_sf(data = Nebraska_county)
```



6. What is the EPSG code of the Counties dataset? Is this a geographic or a projected coordinate reference system? (In other words, does this CRS use angular or planar coordinate units?) To what datum is this CRS associated? (Tip: lookup the EPSG code on <https://epsg.io> or <https://spatialreference.org>)

ANSWER: The EPSG code is 4269, which is the NAD83 reference system. This is a geographic coordinate reference system that uses angular coordinate units.

Read in gage locations csv as a dataframe, then display the column names it contains

Next we'll read in some USGS/NWIS gage location data added to the **Data/Raw** folder. These are in the **NWIS_SiteInfo_NE_RAW.csv** file. (See **NWIS_SiteInfo_NE_RAW.README.txt** for more info on this dataset.)

7. Read the **NWIS_SiteInfo_NE_RAW.csv** file into a standard dataframe, being sure to set the **site_no** field as well as other character columns as a factor.
8. Display the structure of this dataset.

```
#7. Read in gage locations csv as a dataframe
gage_locations<-read.csv(here('Data/Raw/NWIS_SiteInfo_NE_RAW.csv'),
                        stringsAsFactors = TRUE ,
                        colClasses = c('site_no' = 'factor'))

#8. Display the structure of the dataframe
str(gage_locations)
```

```
## 'data.frame':    175 obs. of  7 variables:
## $ site_no      : Factor w/ 175 levels "06453600","06454100",...: 1 2 3 4 5 6 7 8 9 10 ...
## $ station_nm   : Factor w/ 175 levels " BIG SANDY CR AT ALEXANDRIA, NE",...: 117 85 87 70 86 88
## $ site_tp_cd   : Factor w/ 1 level "ST": 1 1 1 1 1 1 1 1 1 1 ...
## $ dec_lat_va   : num  42.8 42.4 42.9 42.7 42.8 ...
## $ dec_long_va  : num  -98.2 -103.8 -100.4 -99.7 -99.3 ...
## $ coord_acy_cd : Factor w/ 6 levels "1","5","F","H",...: 6 6 6 6 6 6 6 6 6 6 ...
## $ dec_coord_datum_cd: Factor w/ 1 level "NAD83": 1 1 1 1 1 1 1 1 1 1 ...
```

9. What columns in the dataset contain the x and y coordinate values, respectively?

> ANSWER: > dec_long_va contains the x and dec_lat_va contains the y

Convert the dataframe to a spatial features (“sf”) dataframe

10. Convert the dataframe to an sf dataframe. * Note: These data use the same coordinate reference system as the counties dataset

11. Display the structure of the resulting sf dataframe

```
#10. Convert to an sf object
gage_loc_sf <- st_as_sf(gage_locations,
  coords = c('dec_long_va', 'dec_lat_va'),
  crs = 4269
)
```

```
#11. Display the structure
```

```
str(gage_loc_sf)
```

```
## Classes 'sf' and 'data.frame':  175 obs. of  6 variables:
## $ site_no      : Factor w/ 175 levels "06453600","06454100",...: 1 2 3 4 5 6 7 8 9 10 ...
## $ station_nm   : Factor w/ 175 levels " BIG SANDY CR AT ALEXANDRIA, NE",...: 117 85 87 70 86 88
## $ site_tp_cd   : Factor w/ 1 level "ST": 1 1 1 1 1 1 1 1 1 1 ...
## $ coord_acy_cd : Factor w/ 6 levels "1","5","F","H",...: 6 6 6 6 6 6 6 6 6 6 ...
## $ dec_coord_datum_cd: Factor w/ 1 level "NAD83": 1 1 1 1 1 1 1 1 1 1 ...
## $ geometry     :sfc_POINT of length 175; first list element:  'XY' num  -98.2 42.8
## - attr(*, "sf_column")= chr "geometry"
## - attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",...: NA NA NA NA NA
## ..- attr(*, "names")= chr [1:5] "site_no" "station_nm" "site_tp_cd" "coord_acy_cd" ...
```

```
st_crs(gage_loc_sf)$epsg
```

```
## [1] 4269
```

12. What new field(s) appear in the sf dataframe created? What field(s), if any, disappeared?

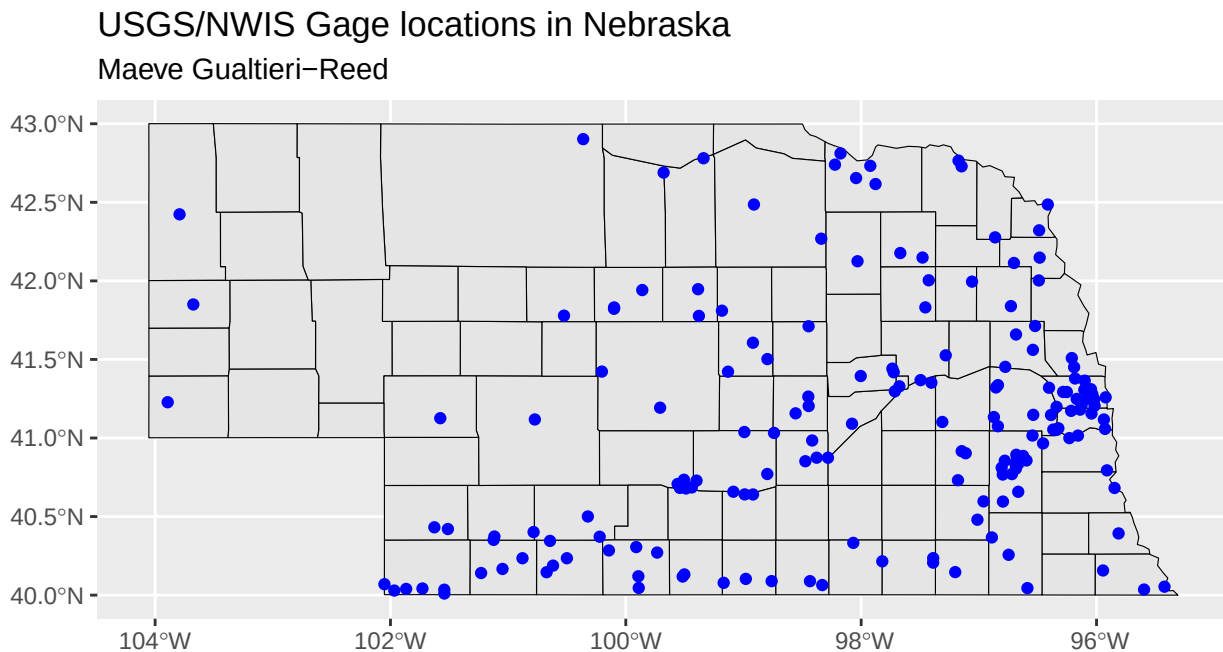
ANSWER: A new geometry field appears and the longitude and latitude columns disappear in the new sf dataframe.

Plot the gage locations on top of the counties

13. Use `ggplot` to plot the county and gage location datasets.

- Be sure the datasets are displayed in different colors
- Title your plot “USGS/NWIS Gage Locations in Nebraska”
- Subtitle your plot with your name

```
#13. Plot the gage locations atop the county features
ggplot() +
  geom_sf(data = Nebraska_county, color = "black")+
  geom_sf(data = gage_loc_sf, color = "blue")+
  ggtitle("USGS/NWIS Gage locations in Nebraska")+
  labs(subtitle="Maeve Gualtieri-Reed")
```



Read in the gage height data and join the site location data to it.

Lastly, we want to attach some gage height data to our site locations. I’ve constructed a csv file listing many of the Nebraska gage sites, by station name and site number along with stream gage heights (in meters) recorded during the recent flood event. This file is titled `NWIS_SiteFlowData_NE_RAW.csv` and is found in the `Data/Raw` folder.

14. Read the `NWIS_SiteFlowData_NE_RAW.csv` dataset in as a dataframe

- Pay attention to which fields should be imported as factors!

15. Show the structure of the dataframe.
16. Join our site information (already imported above) to these gage height data
 - The `site_no` and `station_nm` can both/either serve as joining attributes
 - Construct this join so that the result only includes records features where both tables have data (N=136)
17. Show the column names of this resulting spatial dataframe
18. Show the dimensions of the resulting joined dataframe

#14. Read the site flow data into a data frame

```
site_flow<-read.csv(here('Data/Raw/NWIS_SiteFlowData_NE_RAW.csv'),
                    stringsAsFactors = TRUE,
                    colClasses = c('site_no' = 'factor'))
site_flow<-site_flow %>%
  separate(date, into = c("Date", "Time"), sep = " ")
site_flow$Date <- ymd(site_flow$Date)
```

#15. Show the structure of the dataframe

```
str(site_flow)
```

```
## 'data.frame':   152 obs. of  5 variables:
## $ site_no      : Factor w/ 133 levels "06453600","06453620",...: 1 2 3 3 4 5 6 6 7 8 ...
## $ station_nm   : Factor w/ 133 levels "Antelope Creek at 27th Street at Lincoln, Nebr.",...: 89 61 65 65 65 65 65 65 65 65 ...
## $ Date         : Date, format: "2019-03-20" "2019-03-20" ...
## $ Time         : chr  "14:15:00" "14:15:00" "14:15:00" "14:15:00" ...
## $ gage_ht      : num  12.58 19.96 3.61 3.63 11.38 ...
```

#16. Join the flow data to our NWIS gage location spatial dataframe

```
gage_flow_join <- gage_loc_sf %>%
  left_join(
    site_flow,
    by = c("site_no" = "site_no")
  )
gage_flow_join<-gage_flow_join %>%
  na.omit()
```

#17. Show the column names in the resulting spatial dataframe

```
colnames(gage_flow_join)
```

```
## [1] "site_no"          "station_nm.x"      "site_tp_cd"
## [4] "coord_acy_cd"     "dec_coord_datum_cd" "station_nm.y"
## [7] "Date"             "Time"              "gage_ht"
## [10] "geometry"
```

#18. Show the dimensions of this joined dataset

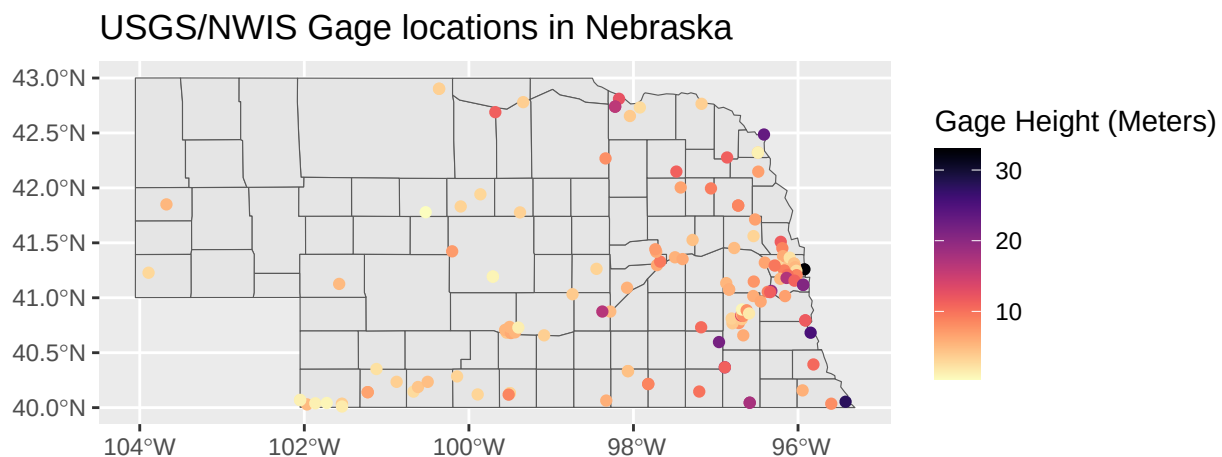
```
dim(gage_flow_join)
```

```
## [1] 136 10
```

Map the pattern of gage height data

Now we can examine where the flooding appears most acute by visualizing gage heights spatially. 19. Plot the gage sites on top of counties (using `mapview`, `ggplot`, or `leaflet`) * Show the magnitude of gage height by color, shape, other visualization technique.

```
#Map the points, sized by gage height  
ggplot() +  
  geom_sf(data = Nebraska_county) +  
  geom_sf(data = gage_flow_join, aes(color = gage_flow_join$gage_ht)) +  
  scale_color_viridis(option = "magma", direction = -1) +  
  ggtitle("USGS/NWIS Gage locations in Nebraska") +  
  labs(color = "Gage Height (Meters)")
```



SPATIAL ANALYSIS

Up next we will do some spatial analysis with our data. To prepare for this, we should transform our data into a projected coordinate system. We'll choose UTM Zone 14N (EPSG = 32614).

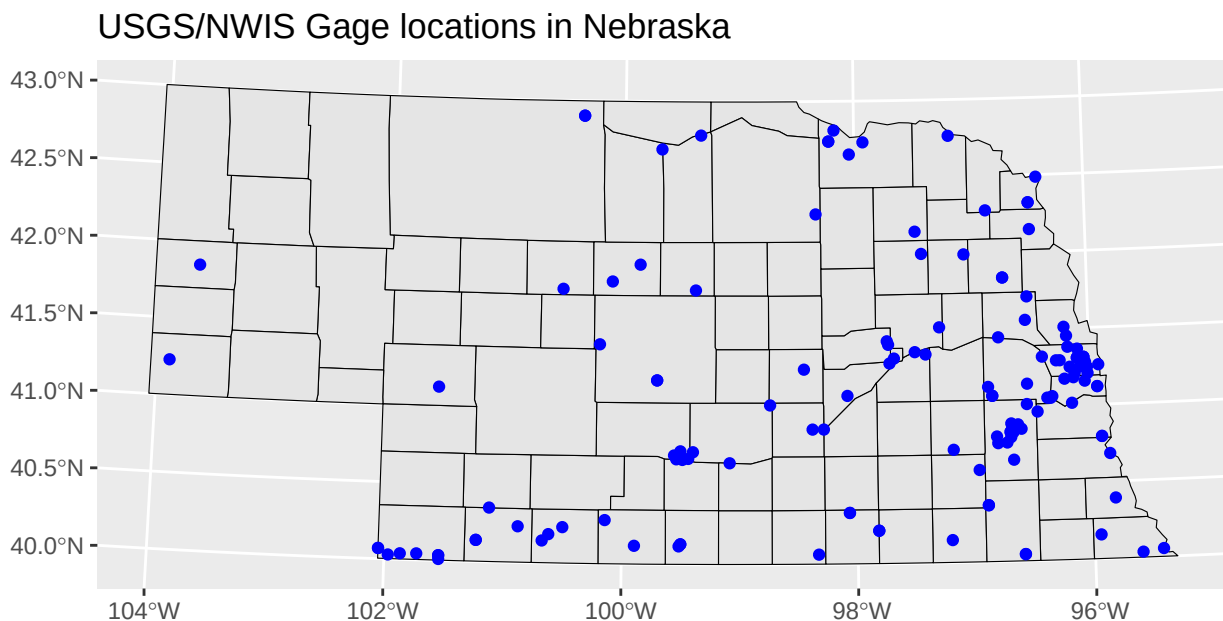
Transform the counties and gage site datasets to UTM Zone 14N

20. Transform the counties and gage sf datasets to UTM Zone 14N (EPSG = 32614).

21. Using ggplot, plot the data so that each can be seen as different colors.

```
#20 Transform the counties and gage location datasets to UTM Zone 14
counties_utm<- st_transform(Nebraska_county,32614)
gage_utm<- st_transform(gage_flow_join, 32614)

#21 Plot the data
ggplot() +
  geom_sf(data = counties_utm, color = "black")+
  geom_sf(data = gage_utm, color = "blue")+
  ggtitle("USGS/NWIS Gage locations in Nebraska")
```



Select the gages falling within a given county

Now let's zoom into a particular county and examine the gages located there. 22. Select Washington county from your projected county sf dataframe 23. Select the gage sites falling within that county to a new spatial dataframe 24. Select the gage sites within 20km of the county to a new spatial dataframe 25. Create a plot showing (each symbolized distinctly): * all Nebraska counties, * the selected county, * the gage sites in that county, * and the gage sites within 20 km of the county

```
#22 Select Washington county to a new spatial dataframe
washington_county <- counties_utm %>%
  filter(NAME=="Washington")
```



```

#23 Spatially select gages within the selected county to a new spatial dataframe
washington_gages <- gage_utm %>%
  st_filter(washington_county, .predicate = st_intersects)

#24 Spatially select gages within 20k of the selected county to a new spatial dataframe
#created a buffer of 20k around washington county
washington_county_buffered <- st_buffer(washington_county, 20000)

washington_gages_buffered <- gage_utm %>%
  #found all gages that intersect with the buffered county
  st_filter(washington_county_buffered, .predicate = st_intersects) %>%
  #created a new column to distinguish sites within the county and those in the
  #buffer
  mutate(Location = ifelse(site_no == "06610705" | site_no == "06610710",
    "In Washington County", "< 20km from Washington County"))

#25 Plot
ggplot() +
  geom_sf(data = Nebraska_county, color = "grey")+
  geom_sf(data = washington_county, color = "black")+
  geom_sf(data = washington_gages_buffered, size = 0.75, aes(color = washington_gages_buffered$Location))+
  ggtitle("USGS/NWIS Gage locations In and Near Washington County, Nebraska")+
  labs(color = "Gage Location")+
  theme(legend.position = "bottom")

```

USGS/NWIS Gage locations In and Near Washington County, Nebraska

